

Practical Covert Channel across Isolated Browser Instances via GPU Command Queue Contention

Jinhong Liu*, Zifeng Kang*†, Song Li‡ and Yinzhi Cao§
The State Key Laboratory of Blockchain and Data Security, Zhejiang University;
Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security
†*Beijing University of Posts and Telecommunications*
§*Johns Hopkins University*
{*12421030, songl*}*@zju.edu.cn, zifengkang@bupt.edu.cn, yinzhi.cao@jhu.edu*

2026 IEEE Symposium on Security and Privacy (SP) | 979-8-3315-6065-2/26/$31.00 ©2026 IEEE | DOI: 10.1109/SP63933.2026.00254

***Abstract*—Web storages, e.g., cookies, are isolated across different browser modes (e.g., normal vs. incognito) and browsers (e.g., Chrome vs. Safari) to ensure user privacy. Despite such browser-level isolation, researchers have found covert communication channels between different modes and browsers, which can break such a security mechanism. However, existing covert communications are impractical and unreliable in the real world scenarios, where noise is prevalent. Furthermore, multiple prior covert channels only support cross-mode—not cross-browser—communications, because the relevant resources are shared within a single browser.**

In this paper, we discover a novel covert channel due to GPU command queue contention and then design a framework, called C^{3}F, for both cross-mode and cross-browser covert communication. C^{3}F negotiates an adaptive communication speed between the sender and the receiver and then facilitates the covert communication. We evaluated C^{3}F in real-world settings by deploying it to crowd-sourced workers on Amazon Mechanical Turk, simply asking them to visit two websites without imposing additional constraints. Our evaluation shows that covert communication achieves a 100% accuracy once the data transmission is completed, with an overall completion rate of 92.81% across 1,434 AMT experiments. So far, Firefox, Safari and Tor developers have confirmed our attack and are working with us on practical defenses.

1. Introduction

Web applications, just like desktop applications with process-level isolation, are also isolated from each other based on security mechanisms enforced by web browsers. For example, modern browsers often support both normal and incognito modes with separate user profiles—which contain isolated data storage for cookies and browsing history—for privacy-preserving web browsing. Some privacy-aware users also adopt different web browsers, e.g., Firefox vs. Chrome, for better isolation.

While inter- and intra-browser isolation exists for privacy-aware web users, people have discovered many covert channels [1]–[6] that bypass such isolation mechanisms and allow communication across isolated browser instances. However, none of them are practical and tested in real-world scenarios where noise is prevalent for reliability. For example, Pool-Party [1] exploits the contention between browser resource pools, and Loophole [2] exploits the shared event loops for a covert channel. Both channels only exist for cross-mode, but not cross-browser communication. For another example, SideR [3] from Wu et al. allows cross-browser communication via rendering resource contention with no hardware constraint. However, SideR requires full GPU or CPU usage so that the performance of the two communicating parties can be influenced by each other. The need for full GPU or CPU usage leads to a limited bandwidth, e.g., the transmission of 512 bits takes about 30 minutes. That is, the covert communication of SideR, unlike their history sniffing, is not reliable or practical in the real-world environment.

In this paper, we discover a novel covert channel, which exists due to GPU command queue contention. The channel utilizes a feature, called OpenGL synchronization [7], which ensures that given commands from an issuer are fully executed. The sender in the channel issues a heavy-weight command, for example, a drawing with a complex calculation, and then waits for its completion using a synchronous command. At the same time, the receiver issues many lightweight, synchronous commands, e.g., drawing a random point and reading it back. One, and only one, of these many issued by the receiver—which we call probes—will wait for completion of the sender's command and thus delayed, leading to a contention; the rest will be scheduled before or after the sender's command, perceiving minimum delays. Since the sender can control the complexity of the command, the command's rendering time conveys one or more bits of information. Furthermore, multiple commands will convey a stream of information over time.

We further design a cross-browser and cross-mode covert communication framework, called C^{3}F, built upon the covert channel to address two real-world deployment

* *Jinhong Liu and Zifeng Kang contributed equally to this paper.*
‡ *Song Li is the corresponding author.*

© 2026, Jinhong Liu. Under license to IEEE.

DOI 10.1109/SP63933.2026.00254
Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

challenges. First, the delay issued by the sender and perceived by the receiver varies between different browsers, machines, and graphic cards, making it challenging for the communicating parties to reliably distinguish between a bit one and a bit zero. To overcome this, C^{3}F utilizes a handshake stage for both parties to negotiate a proper threshold tailored to their specific system configurations for reliable and faster communication. Second, real-world machines often have other rendering tasks running at the same time, which introduces noise and thus influences communication. Therefore, C^{3}F adopts multiple alignment and error correction mechanisms (i.e., protocols and checksums) to ensure the accuracy of the information received.

We implemented an open-source prototype of C^{3}F at this repository [8]. An online demo at the website [9] is also available. Furthermore, we evaluated C^{3}F under three different conditions: real-world scenarios, ideal environments, and settings with defense mechanisms in place. In the real-world setting, we instructed workers hired from Amazon Mechanical Turks (AMT) [10] to visit demo websites with different browsers or different modes. The evaluation demonstrates that C^{3}F successfully completed data transmission on 92.81% of the tested cases, covering a broad spectrum of PC devices, including Intel, NVIDIA, AMD, and Apple GPUs, as well as some virtual GPUs. Moreover, when the transmission completes, the accuracy reaches 100%. The overall average bandwidth in real-world settings is 6.82 bps. In the real world, a website's cookies have a median size of approximately 36 bytes [11], requiring about 40 seconds for C^{3}F to transmit. An attacker can therefore achieve cookie synchronization within the average time a user spends on a page (52 seconds) [12]. The broad applicability gives C^{3}F an advantage in universality over hardware-specific channels [4]–[6], [13], [14]. Reliable high accuracy and adequate bandwidth make C^{3}F more practical than Pool-Party [1], Loophole [2] and SideR [3]. In the ideal setting, we compared C^{3}F against two baseline methods for cross-browser and cross-mode communication on a local device. The results demonstrate that C^{3}F significantly outperforms both baselines in terms of transmission accuracy and bandwidth. Finally, in the defense setting, we evaluated C^{3}F under the Tor Browser and a fuzzy-time-based defense deployed in Chrome. Despite the reduced transmission bandwidth caused by these countermeasures, C^{3}F remained effective in successfully transmitting data between two parties.

We responsibly disclose our findings to all the affected web browsers. So far, Firefox, Safari and Tor developers have confirmed our attack and are working with us on practical defenses.

To summarize, we make the following four main contributions in this paper.

- We present a novel covert channel based on GPU command queue contention and detail its underlying principle, enabling communication between isolated browser instances.
- We design and implement C^{3}F, a framework for practical cross-browser and cross-mode communication, achieving 100% transmission accuracy even in real-world settings. Meanwhile, we have open-sourced C^{3}F at an anonymous repository and provided a live demo on an anonymous website.
- The evaluation of C^{3}F on AMT workers' devices indicates that the GPU command queue channel is compatible with a broad range of hardware platforms and mainstream browsers.
- Experiments on our local devices further demonstrate that C^{3}F higher transmission accuracy than the two baselines while maintaining substantially high bandwidth. Furthermore, even the Tor Browser with a fuzzy-time-based defense can still maintain 100% accuracy by decreasing bandwidth to an appropriate level.

2. Related Work

In this section, we discuss related attacks and defense.

2.1. Browser Covert Channels

Researchers have identified various side and covert channels that can be leveraged to perform attacks on browsers and websites including browser fingerprinting [15]–[25], website fingerprinting [2], [26]–[32], keystroke timing attacks [33]–[35], cross-site cookie synchronization [1]–[3], information leakage [5], [36], [37], history sniffing [4], [38]–[41], cryptographic key extraction [5], and cross-origin content inference [5], [42]–[44].

While some works do explore the possibility of leveraging their discovered channels for covert communication between browser tabs, those channels are not practical and reliable for both cross-mode and cross-browser communication. For instance, Genkin et al. [5] identify a side channel based on contention of the CPU cache to extract cryptographic secret keys, and then use this contention to establish a covert channel for communication between browser tabs. Although they report a high transmission rate under ideal conditions, their channel does not support cross-browser communication, given that the channel is tied to the Chrome browser. Furthermore, there is no evidence that this channel is practical and reliable (i.e., with high transmission accuracy and high bandwidth) in real-world scenarios where noise is prevalent. Another example is the channel proposed by Lipp et al. [33], originally designed for keystroke timing attacks. While theoretically applicable for cross-browser covert communication, this channel has not been validated in noisy environments and lacks reliability due to its unidirectional nature and absence of error correction mechanisms. A third example is the channel discovered by Rokicki et al. [6], which is conceptually extendable to support cross-browser covert communication but has a strong threat model by assuming attacker-victim co-location on the same core. Moreover, this channel relies on simultaneous multi-threading (SMT), whose applicability to non-Intel architectures has yet to be demonstrated. Following up on that work, Rokicki et al. [45] proposed another port contention channel that avoids

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

TABLE 1: Comparison of C^3F with three closely related covert channels from prior work.

Work (Covert Channel)	Supported Communication†			Acc.	Bandwidth
	X-Sites	X-Modes	X-Browsers		
SideR [3] (Page Rendering)	✓	✓	✓	~50%	~0.3 bps
Loophole [2] (Event Loops)	✓	✓	✗	~60%	~5.0 bps
Pool-Party [1] (Resource Pools)	✓	✗	✗	~90%	~6.0 bps
C^3F (GPU Command Queue)	✓	✓	✓	100%*	~8.0 bps

† X-Sites stands for cross-site, X-Modes for cross-mode (i.e., normal to incognito) and X-Browsers for cross-browser.
* C^3F achieves 100% accuracy upon transmission completion, e.g., in 92.81% of AMT experiments.

SMT, but it is only demonstrated for CPU fingerprinting, not for cross-browser communication. Compared to these prior works, our proposed framework, called C^3F, is not only practical and reliable for both cross-mode and cross-browser communication with clear and direct evidence, but also compatible with any underlying hardware.

Microarchitectural covert channels enable cross-process communication and could theoretically be extended to cross-browser covert channels, as these components are shared without isolation by any abstraction layer. For this channel, prior work achieved 45 KBps using Intel CPU caches [13] and 4 Mbps using Nvidia General Purpose GPUs (GPGPUs) [14], both with zero errors. Despite high bandwidth and error-free performance, their applicability is limited to hardware-specific designs. To the best of our knowledge, the underlying principle has not been validated on Apple or AMD CPUs, or on Intel or AMD GPUs. Porting them requires significant efforts. One reason is the need for complex reverse engineering of undocumented algorithms, including cache mapping [13] and GPU scheduling [14]. The other reason is the fundamental microarchitectural differences—for example, inclusive caches on Intel vs. non-inclusive on AMD [4].

From all the prior works, we identify three that are closely related to our work: SideR [3], Loophole [2] and Pool-Party [1], all of which leverage their own discovered channels for covert communication between isolated browser instances. We summarize the communication types that C^3F and these three works support in Table 1, and discuss the works in details.

- SideR. Wu et al. [3] discovered a rendering contention channel that measures frame rendering times to detect co-rendering events. They further designed and implemented a framework called SideR, which launched four different attacks using the contention channel, including cross-browser and cross-mode covert communication. However, SideR's dependency on extreme resource contention makes it both unreliable and impractical in real-world environments, where system resources are rarely fully consumed.
- Loophole. Vila et al. [2] demonstrated that shared event loops are vulnerable to side-channel attacks. By monitoring the usage patterns of event loops in Chrome browser, they showed how a spy process could establish cross-site and cross-mode covert communication. However, because the event loops are not shared across different browsers, this attack by nature fails to establish cross-browser communication. Meanwhile, the performance of their tool with the presence of noise heavily relies on hyper-parameter settings, making the tool less reliable in the real world.
- Pool-Party. Snyder et al. [1] identified a class of covert channels in browsers called "pool-party", which exploit limited but unpartitioned resource pools. These attacks can pass cookies and identifiers across site boundaries, making them practical for cross-site tracking. Nonetheless, the resource pools are not shared across different browser instances, and thus becoming infeasible for cross-mode or cross-browser covert communication.

As a comparison, the covert channel that we discovered, which stems from GPU command queue contention, is shared across different browser instances, so it supports covert communication of all three types: cross-site, cross-browser and cross-mode. Furthermore, the framework, C^3F, that we designed and implemented based on the discovered channel, is reliable and practical in real-world scenarios.

2.2. General Side or Covert Channels

More generally, side and covert channels have been discovered in various resources such as CPU [13], [46]–[53], GPU [54]–[62], virtual machines [63]–[65], LLC [4], [52], [64], [66]–[68], storage [69]–[71], LED [72]–[75], speaker [76]–[79] and other categories [80]–[104]. However, none of those prior works discovered a covert channel based on GPU command queue contention, nor did they focus on establishing a practical covert channel for communication between isolated browser instances.

It is worth noting that several GPU-based covert channels proposed in prior work rely on mechanisms that differ from the covert channel we discovered. We detail the differences below.

- SideR [3]. SideR uses a lock-like mechanism that requires 100% GPU utilization as opposed to a flexible GPU utilization in C^3F.
- General Purpose GPU (GPGPU)-based covert channel [14]. GPGPU-based covert channel relys on contention of Streaming Multiprocessor (SM) resources, which may face SM co-residency issues. In contrast, C^3F creates resource contention by using synchronization commands rather than demanding specific GPU resources.
- GPU cache-based channels [54], [59]. GPU cache-based channels rely on the hardware-specific cache access mechanism as opposed to C^3F's general command queue provided by OpenGL, a standard graphics library.

2.3. Defenses against Covert Channels

Defenses against covert channels include a variety of strategies. For example, timing mitigations introduce either fuzzy time [4], [105]–[108], which adds random noise or time delay, or deterministic time [109], [110], which ensures

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

TABLE 2: Categorization of GLSL commands used in the GPU command queue contention channel.

Category	Command	Description
Drawing	`drawArrays`	Renders primitives using array data
	`drawElements`	Renders primitives using indices
Syncing	`finish`	Blocks until all GPU commands finish
	`convertToBlob`	Converts framebuffer to a blob
	`readPixels`	Reads pixel data from the framebuffer

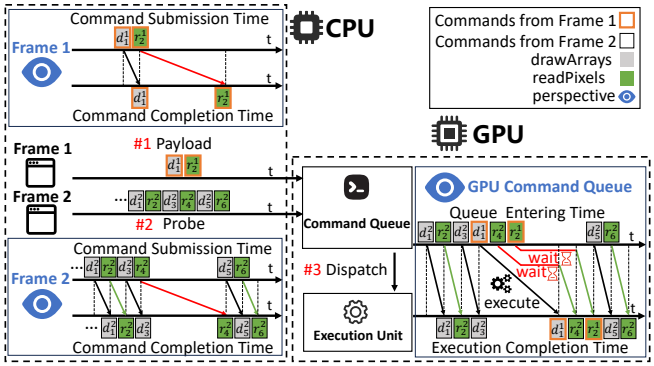


Figure 1: An Illustration of the Command Queue Contention Channel using two Isolated Web Frames.

consistent timing, to disrupt attackers' timing measurements. Resource partitioning [111]–[114] is another key approach, isolating critical resources like CPU caches and memory between different browser tabs. Other defense mechanisms include restricting access to high-resolution timers to reduce the precision of timing attacks [4], [115], site isolation [116], rate limiting [31] and system-level defenses [117]–[120].

3. GPU Command Queue Contention Channel

In this section, we describe and evaluate the GPU command queue contention channel, which serves as the foundation of our C^{3}F framework, and finally present our threat model.

3.1. GPU Command Queue Contentions

1) GPU Command Queue. A GPU command queue is a data structure that manages and orders commands that the CPU issues to the GPU. At a high level, GPU command queue contention refers to a phenomenon in the GPU command queue, where commands issued by one process may be delayed due to commands issued by another. Normally, processes do not perceive such delays because the CPU issues *drawing commands* asynchronously without waiting for the completion of these commands. Nonetheless, the OpenGL synchronization mechanism [7] enables the observations of such delays by providing *syncing commands* to let the CPU wait. Table 2 shows these two categories of commands: drawing commands , like `drawArrays`, are asynchronous and returned immediately, but syncing commands will wait for the completion of the issued commands. Some syncing commands, such as `finish`, are explicit, meaning their sole purpose is to wait for the completion of preceding commands. Others, like `readPixels`, are implicit: while their function does not explicitly involve waiting, they inherently do so because they must wait for the results of preceding drawing commands to be available in the framebuffer before retrieving data.

2) GPU Command Queue Contention. We describe contentions on the GPU command queue in Figure 1. Two web frames, isolated in different modes or browsers, are performing rendering tasks and then retrieving the resulting data. The first frame ("Frame 1") issues a heavyweight drawing command followed by a syncing command (e.g., a `drawArrays` followed by a `readPixels`). Here we use the notation d_x^y to denote a drawing command (`drawArrays`), and r_x^y to denote a syncing command (`readPixels`), where y is the frame index and x is the command's order within that frame. The second frame ("Frame 2") continuously issues many command pairs consisting of a lightweight drawing command followed by a syncing command. Each pair is called a probe. From the GPU command queue's perspective, when commands d_3^2, d_1^1 and r_4^2 are in the queue, r_4^2 needs to wait for d_1^1's completion to retrieve the result data of d_3^2, and therefore the completion time of r_4^2 is delayed, leading to a contention. From the Frame 2's perspective, the syncing command r_4^2 experiences a delay nearly equal to the actual rendering time of d_1^1, due to the contention in the GPU command queue.

3) Contention Channel. Based on the contention between two isolated frames, we observe a channel for covert communication: A frame like Frame 1, which serves as the sender, conveys different information by altering the *payload*, i.e., the expected rendering time, of the drawing command. For example, a small payload, which is a lightweight rendering task, represents a value zero, a larger payload as a value one, an even larger payload as a value two, and so on. Each value is a signal, and it can be converted into the corresponding bit information. Furthermore, multiple commands issued by the sender can convey a continuous stream of information over time. Then, another frame like Frame 2, which serves as the receiver, can decode the embedded information based on the perceived payload size.

4) Threat Model. We assume an attacker can inject malicious code into two webpages that are simultaneously open in the user's environment, e.g., via compromised or malicious third-party content—including advertising scripts, analytics services, or shared libraries—that is embedded in otherwise benign websites. These two pages may exist in different browser contexts:

- Cross-mode. Both pages are opened in the same browser but in different modes, i.e., normal and incognito. Although these modes are isolated at the storage and process levels, they may still share access to GPU resources.
- Cross-browser. The pages are opened in different browsers, e.g., Chrome and Safari, on the same device.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

This reflects a common real-world setting where users simultaneously open multiple browsers for different purposes, e.g., Chrome for personal use and Safari for work, without expecting covert communication between them.

3.2. Evaluation Results of the Channel

We first describe our experimental setup. Then, we present our channel's experimental evidence and performance results.

1) Experimental Setup. We design two experimental settings to evaluate the discovered contention channel.

First, we measure the delays perceived by the sender and the receiver to prove the existence of the contention. Specifically, we perform the tasks described in Section 3.1 for both frames and measure the perceived delays at Frame 2, i.e., the receiver. We also continuously add payloads (i.e., many d_1^1 and r_2^1 pairs in Figure 1) at Frame 1 (sender), and record the probe delays at Frame 2 (receiver).

Second, we tested the performance of this contention channel for data transmission under different conditions so that it can motivate our design of C^{3}F. Specifically, we transmitted a random 1,024-bit sequence using the Chrome browser for both the sender and receiver, repeating each test five times for every radix and noise level, and calculated the median and Median Absolute Deviation (MAD) for the results. Our experiment measures the edit distance, also known as the Levenshtein distance, between the sent and received messages, to indicate transmission inaccuracies through the channel.

The experiments were conducted on a Windows 11 64-bit device equipped with a 13th Gen Intel® Core™ i9-13900HX CPU at 2.20 GHz, 16 GB of RAM, and a NVIDIA GeForce RTX 4060 Laptop GPU.

2) Experimental Evidence of the Contention. We describe our experimental evidence for the discovery of this contention channel from two aspects: on six different web browsers, and in a short timing window on the Chrome browser.

Figure 2 shows the probe delay (y-axis) at Frame 2 on six popular real-world browsers when the payload (x-axis) of the drawing command at Frame 1 increases. There are two key observations from Figure 2. First, the data points clearly form two distinct clusters: one with low probe delays, marked with "Not Delayed", and another with linearly increasing delays, marked with "Delayed". The non-delayed cluster occurs when probes from Frame 2 do not encounter any contention in the GPU command queue. Second, within the delayed cluster, the probe delay increases proportionally with the payload duration, confirming the presence of GPU command queue contention across real-world browsers.

Next, we illustrate how contention manifests over time by visualizing the timing of sender payloads and receiver probes in a short execution window. This experiment was conducted on the Chrome browser running on the same Windows device. Figure 3 shows the recorded delays for

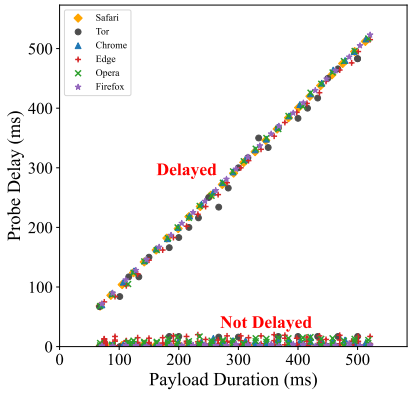


Figure 2: Experimental evidence of GPU command queue contention in the real world: probe delay versus payload duration on six popular browsers. The data points form two distinct clusters: a linearly increasing cluster indicating delayed probes due to the contention, and a near-zero cluster representing non-delayed probes where no contention occurred.

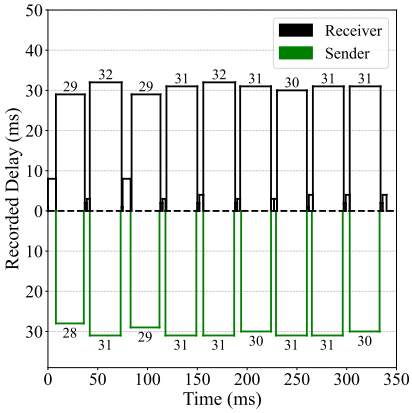
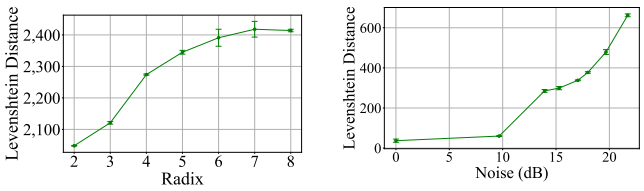


Figure 3: Recorded delays of probes from the receiver (black) and payloads sent by the sender (green) over elapsed time, tested on Chrome. Tall black bars near 30 ms indicate the receiver's delayed probes, which exactly align with the payload delays observed by the sender. In contrast, short black bars under 10 ms correspond to non-delayed probes, during which the sender issues no tasks.

probes from the receiver (top, black) and the delays for payloads sent by the sender (bottom, green). It indicates a clear correlation between the two: when the sender introduces a delay (green bars near 30 ms), the corresponding probe at the receiver is consequently delayed (black bars near 30 ms), demonstrating contention in the GPU command queue. In contrast, when the sender is not sending any tasks, the probe delay remains low (under 10 ms), indicating that there is no contention.

3) Evaluation Results of Channel's Performance. In this part, we design two tests to evaluate the performance of

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.



(a) Radix vs. Edit Distance. (b) Noise vs. Edit Distance.

Figure 4: Results of two tests on the channel. (a): The impact of radix changes for noise-free data transmission in the data radix test on the raw channel. (b): The impact of noise level in the noise intensity test on the raw channel.

the channel for data transmission: data radix test and noise intensity test.

Our first test is to understand how much information can be transmitted in one payload from the sender. Specifically, we used different rendering times as the radix. For example, in a radix-eight data transmission test, we used rendering tasks with eight distinct rendering times, representing values from zero to seven. Additionally, we ensured that the bandwidth remained consistent during the experiments. Figure 4(a) shows that radix two exhibits the lowest Levenshtein distance and radix eight has the highest. This is because a smaller radix requires fewer distinct rendering times, and thus reduces the likelihood of errors.

Our second test is to understand how noise influences the channel's performance. To represent noise in real-word scenarios such as page loading, we leveraged an independent thread to send rendering tasks with random rendering times and measured the noise level. We computed the noise level as the logarithmic ratio of the rendering times for the noise and the probe, with the result expressed in decibels (dB). Figure 4(b) shows the results of the noise intensity test. As the noise level increases, the Levenshtein distance also increases, indicating a higher rate of errors in the transmitted data. This occurs because higher noise levels introduce greater perturbations, which have a higher likelihood of confusing the decoding scripts. Therefore, to ensure reliable data transmission through the raw channel, an error correction mechanism needs to be developed and incorporated, which motivates our design of C^{3}F.

4. C^{3}F: Cross-browser Covert Communication Framework

In this section, we introduce C^{3}F, a framework for reliable cross-mode and cross-browser covert communication. We start from describing the overall architecture of C^{3}F (Section 4.1), followed by detailed discussions on each layer of C^{3}F (Section 4.2 to 4.5).

4.1. Overall Architecture

The overall architecture of C^{3}F is shown in Figure 5 with four layers. We describe the layers from top to bottom:

- Session Layer. This layer controls the progress of the session and assigns three Phases, namely Setup, Transmission and Finishing. First, C^{3}F prepares for data transmission, notifies both parties and sets up the session in the Setup Phase. Then, C^{3}F transmits data packets, gets notified of errors and re-transmits packets when errors are present in the Transmission Phase. Finally, C^{3}F ends the transmission and ensures that both parties are aware of the completion in the Finishing Phase.
- Transmission Layer. C^{3}F generates and handles the data packets designated for transmission through the covert channel within this layer. It also performs error checking on these packets and alerts the Session Layer to initiate a re-transmission if any errors are detected.
- Encoding Layer. This layer generates a bit sequence as an intermediate data representation between the Transmission Layer and Raw Channel Layer. For the incoming data packets from the Transmission Layer, C^{3}F encodes them into bit sequences. C^{3}F then assigns corresponding execution time and transmits this timing information to the Raw Channel Layer. For the outgoing rendering time measured by the Raw Channel Layer, C^{3}F utilizes a clustering algorithm to convert the time back to bit sequences, which are then decoded into data packets.
- Raw Channel Layer. This layer directly interacts with the contention channel, where C^{3}F generates payloads and probes as instructed by the Encoding Layer, and measures the actual delay for each task to report to the Encoding Layer.

4.2. Session Layer

We describe how the following three phases ensure reliable data transmission in this layer.

1) Setup Phase. In this phase, C^{3}F establishes the session between the sender and the receiver and computes important parameters, using the protocol shown in Figure 6(a). First, C^{3}F notifies and configures the sender and receiver, and then instructs the sender to compute "task payloads"—the numbers of for loop iterations of a single mathematical operation in a payload task, which determines the rendering time of the task—for both bit-zero and bit-one tasks. Upon completion, the sender sends an SYN (synchronization) signal, which is a short sequence of bits, to notify the receiver. After that, the receiver completes the computation and sends a SYN signal back to the sender. Upon receiving this signal, the sender confirms that the session has been successfully established. We refer to the whole procedure as a "handshake" between the sender and the receiver. Note that the rendering times for the sender and the receiver may differ, due to factors such as the use of different browsers.

Next, we outline the algorithm C^{3}F adopts to determine the task payloads for bit-zero and bit-one rendering tasks for both the sender and receiver. First, C^{3}F measures the actual rendering time for several preset task payloads. Second, C^{3}F performs linear regression to derive a fitted line between the task payload and the actual rendering time.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

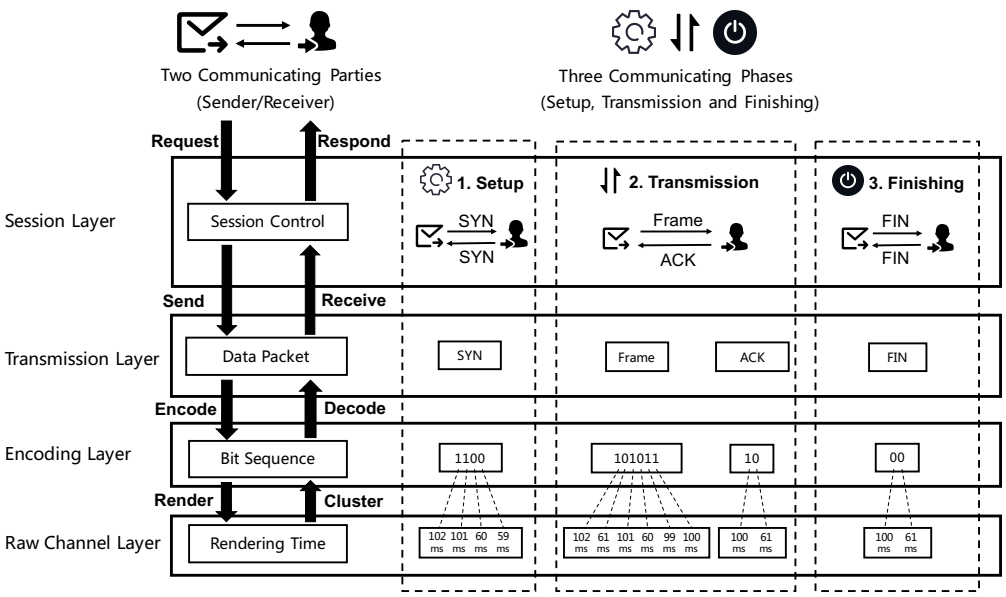
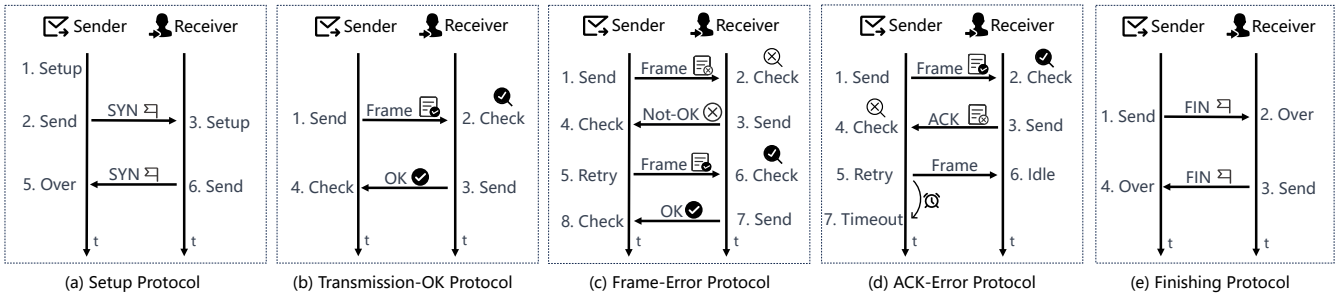


Figure 5: The layer model of C^{3}F. (i) Session Layer controls whole progress of the session by assigning three communicating phases. (ii) Transmission Layer generates, handles and verifies the data packets for transmission. (iii) Encoding Layer acts as an intermediary between the Transmission Layer and the Raw Channel Layer, using bit sequences as intermediate data representation. (iv) Raw Channel Layer directly interacts with the GPU command queue contention channel, generating payloads and probes while measuring delays.



(a) Setup Protocol (b) Transmission-OK Protocol (c) Frame-Error Protocol (d) ACK-Error Protocol (e) Finishing Protocol

Figure 6: Protocols used in the Setup, Transmission and Finishing Phase.

Then, C^{3}F tests different task payloads along the fitted line until the rendering time meets preset criteria: for bit zero, the rendering time must exceed a specific threshold that we call bit-zero threshold; for bit one, the rendering time must exceed another threshold, dubbed offset threshold, plus the previously determined bit-zero task payload. After that, the sender and receiver each use their own minimum task payloads, determined in the above procedure, to represent bits zero and one in the subsequent data transmission. Finally, C^{3}F informs the other layers of the determined task payloads as well as the distinct rendering time to represent bit zero or one.

2) Transmission Phase. In this phase, C^{3}F transmits data using a data frame between both parties, verifies the integrity of the frame, and starts a data re-transmission if an error is detected. We explain the protocols that C^{3}F adopts in this phase as follows. Note that when C^{3}F finishes any protocol below, it switches to the Finishing Phase.

- Transmission-OK. This protocol is used when the transmission is successful and free of errors. As shown in Figure 6(b), upon receiving the data frame with a checksum, the receiver verifies the checksum. If no errors are detected, the receiver responds with an ACK (acknowledgement) frame. The sender then checks if the frame received is an "OK" signal. If confirmed, the transmission is completed.
- Frame-Error. This protocol is used when the Transmission Layer gives an alert of an error data frame. As shown in Figure 6(c), upon receiving a data frame with an error, the receiver cannot verify the checksum. The receiver then responds with a "Not-OK" signal. Once that signal is confirmed by the sender, the sender will re-transmit the data frame until the receiver can verify the checksum of the data frame and the sender can confirm an "OK" signal.
- ACK-Error. This protocol is used when the Transmission Layer gives an alert of an error ACK frame. As shown in Figure 6(d), the receiver can verify the checksum

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

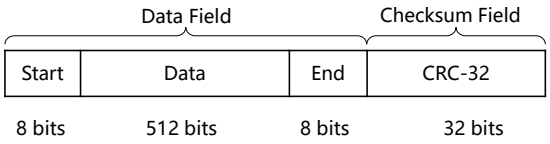


Figure 7: Design of the data frame leveraged by C^{3}F to ensure reliable data transmission. The data field consists of a Start delimiter, the data payload and an End delimiter. The checksum field consists of a 32-bit CRC-32 checksum.

of the data frame, whereas the sender cannot confirm the responded ACK frame, i.e., there is an error when transmitting the ACK frame. The sender will continue re-transmitting the data frame until it reaches a timeout. Meanwhile, the receiver will remain idle since it has already received a correct data frame.

3) Finishing Phase. In this phase, the sender sends a FIN (finishing) message to mark the end of data transmission. The receiver then sends back a FIN message to notify the sender and C^{3}F terminates the communication, as illustrated in Figure 6(e).

4.3. Transmission Layer

We describe how C^{3}F generates and verifies the data packets designated for transmission through the covert channel within this layer.

1) Generating Data Packets. In this layer, when handling incoming traffic, C^{3}F generates different data packets as instructed by the Session Layer, which can be a data frame or an ACK frame.

- Data Frame. As shown in Figure 7, a data frame comprises two fields: a data field and a checksum field. The data field is divided into three segments: a Start delimiter, the data payload and an End delimiter. The Start and End delimiters are fixed eight-bit sequences that mark the start and end of the 512-bit data payload. Then, in the checksum field, C^{3}F appends applies the Cyclic Redundancy Check-32 (CRC-32) algorithm to the data payload, generating the 32 bits of the checksum field.
- ACK Frame. An ACK frame is an eight-bit sequence to indicate different status in the Session Layer. C^{3}F chooses different fixed bit sequences to represent an SYN, OK, Not-OK, or FIN signal.

2) Verifying Checksum. Next, when handling outgoing traffic, C^{3}F verifies checksum on an array of bit sequences received from the Encoding Layer. If the checksum is correct, C^{3}F decodes them to data packets. Specifically, C^{3}F first categorizes the bit sequences by their type, determining whether they correspond to a data frame, an ACK frame, or an SYN or FIN signal, using the session information retrieved from the Session Layer. For a data frame, C^{3}F processes each bit sequence from the received sequence array and follows three steps: i) retrieving each segment from the sequence according to the design depicted in Figure 7; ii) computing CRC-32 checksum on the retrieved data segment; and iii) verifying each segment. For the start and end delimiters, C^{3}F checks if they match fixed sequences. For the checksum field, C^{3}F verifies whether the computed checksum matches that retrieved from the original sequence. Only if the verifications succeed does C^{3}F consider the sequence correct. C^{3}F then counts the number of correct sequences. If the count is zero or more than one, the verification fails, and C^{3}F alerts the Session Layer. That said, the verification process succeeds only if there is exactly one correct sequence. Finally, C^{3}F decodes the correct bit sequence into data packets and sends them to the Session Layer.

Algorithm 1 Clustering Algorithm

```
Input: timeArr[]
Output: bitSequenceArr[]
index ← 0
sortedArr[] ← SORT(timeArr[])
for i ← 0 : sortedArr.length do
    clusterArr[index].push(sortedArr[i])
    if i < sortedArr.length then
        if sortedArr[i + 1] − sortedArr[i] > 3 then
            index+ = 1
for i ← 0 : index do
    clusterMin[i] ← FINDMIN(clusterArr[i])
for i ← 0 : index do
    bitSequenceArr[] ←
        GETSEQUENCE(clusterMin[i])
    return bitSequenceArr[]
```

4.4. Encoding Layer

In this layer, C^{3}F generates bit sequences as an intermediate data representation between Transmission Layer and Raw Channel Layer.

- Incoming Traffic. For the incoming traffic which are data packets sent from the Transmission Layer, C^{3}F encodes data packets to a bit sequence and then to a sequence of rendering time. Specifically, C^{3}F first encodes data packets by selecting predefined bit sequences for an ACK frame or by computing the corresponding bit sequences for each field in a data frame. C^{3}F then converts the bits into a sequence of rendering time, using the appropriate rendering time information provided by the Session Layer to represent bit zero or one. Next, C^{3}F forwards the rendering time sequence to the Raw Channel Layer to perform rendering tasks.
- Outgoing Traffic. For the outgoing traffic which is a rendering time sequence sent from the Raw Channel Layer, C^{3}F clusters it to obtain an array of bit sequences using a clustering algorithm which will be introduced in the following paragraph, and then forwards the array to the Transmission Layer for checksum verification.
- The clustering algorithm. We introduce the algorithm utilized by C^{3}F when handling outgoing traffic in the Encoding Layer, detailed in Algorithm 1. This algorithm processes an array of rendering times from the Raw Channel Layer and outputs an array of bit sequences. Initially,

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

the rendering times are sorted (Line 2). Following this, C^{3}F categorizes these times into clusters (Lines 3-7), ensuring each cluster maintains a small time gap (e.g., 3 ms) from one another at Line 6. In the next step (Lines 8-11), C^{3}F uses the minimum rendering time from each cluster as a threshold to differentiate bit zero from bit one. Consequently, the algorithm generates and outputs an array of bit sequences.

4.5. Raw Channel Layer

In this layer, C^{3}F performs the rendering tasks in the raw covert channel and measures the rendering time.

- Generating rendering tasks and probes with specified rendering time. C^{3}F creates rendering tasks and probes in the vertex shader, a critical stage in the WebGL rendering pipeline. Specifically, to generate a rendering task, C^{3}F repeatedly performs a mathematical operation that executes within a fixed time period, with the number of repetitions determined by the signal from the Encoding Layer. On the other hand, to generate a probe with a typical rendering time of less than two milliseconds, C^{3}F performs a mathematical operation with a small fixed time period only once. Next, C^{3}F forwards the rendering time to the Raw Channel Layer.
- Measuring the actual rendering time. Similar to the progress described in Section 3.1, C^{3}F sends probes to the GPU command queue at relatively small time intervals and measures the actual rendering time. C^{3}F then outputs the results to the Encoding Layer.

5. Implementation

We implemented a prototype of C^{3}F comprising 3,377 lines of JavaScript, which handles the logic for our four-layer design, along with 1,583 lines of Python and 489 lines of HTML to set up the interactive website. The prototype has been deployed on a Linux machine running CentOS 7.9, powered by a 64-bit Intel Xeon Platinum processor, with 5 Mbps bandwidth, 2 vCPUs, and 4 GiB of RAM. We have open-sourced the implementation in this anonymous repository [8] and provided a live demo at this anonymous website [9]. Below, we provide necessary implementation details: (i) We heuristically selected two thresholds (as described in Section 4.2.1) for task payloads. The bit-zero threshold was set to 70 ms, and the offset threshold to 40 ms. (ii) We selected `sin(x)` as the mathematical operation for each iteration when C^{3}F was generating payloads. Note that C^{3}F took the output of `sin(x)` as the new value of `x` in each subsequent iteration. (iii) The delay for each task was computed via `Date.now()`.

6. Evaluation

6.1. Research Questions and Experimental Setup

Research Questions (RQs). We answer the following RQs:

- **RQ1: Performance.** What are the attack success rate, transmission accuracy, and bandwidth of C^{3}F in real-world scenarios with noise?
- **RQ2: Comparison.** Can C^{3}F outperform prior covert channels with their own protocols for cross-browser (RQ2.1) and cross-mode (RQ2.2) covert communication under ideal conditions? What is the Shannon capacity comparison of the raw channels without the influence of their own protocols? (RQ2.3)
- **RQ3: Intermediate Results.** What are the intermediate results, e.g., the task payloads (RQ3.1) for AMT experiments, retransmissions (RQ3.2), and GPU Utilization (RQ3.3)?
- **RQ4: Defense Analysis.** Can existing defense mechanisms mitigate the covert communication facilitated by C^{3}F?

Data Collection. The data collection process is three-fold. First, to evaluate C^{3}F in real-world scenarios with noise, we implemented our C^{3}F prototype on a website and recruited workers from Amazon Mechanical Turk (AMT). We instructed them to open our experimental web pages on their own computers. This allowed us to collect data from their devices for our evaluation. In total, we collected results from 1,524 experiments conducted by 232 unique workers in two weeks. Note that the backend code for the experiment is deployed on the same Linux device that we describe in Section 5, and the transmitted sequences are 512-bit random bitstrings.

Second, to evaluate C^{3}F under ideal conditions, i.e., with little or no noise, we implemented our C^{3}F prototype and two baselines on two local devices. One of them is the same Windows device that we describe in Section 3.2 and the other is a MacBook Pro device with an Apple M1 chip (ANGLE Metal Renderer: Apple M1), used for Safari-involved experiments. The transmitted sequences are 512-bit random bitstrings, except in Safari-involved experiments, where they are 256 bits long due to SideR's inability to transmit 512-bit sequences when Safari is involved. Each experiment was repeated ten times. In total, we collected results from 380 local experiments.

Baselines. We compare C^{3}F with two baselines:

- **SideR** [3]. We adopted the original source code of SideR, with a fix for a minor bug related to ASCII code computation that we identified. We then compared C^{3}F with SideR for both cross-browser and cross-mode communications.
- **Loophole** [2]. Given that Loophole does not support cross-browser communication, we only compared C^{3}F with Loophole for cross-mode communication. We adopted the original source code of Loophole and set its parameters to optimal values. Specifically, the parameters comprised $t_b = 200$ ms and $N = 350$ as specified by the authors, and a threshold value of 15, which was selected heuristically based on our experiments.
- **Pool-Party** [1]. Given that Pool-Party does not support cross-browser or cross-mode communication, we only

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

compared C^{3}F with Pool-Party for the Shannon capacity. We adopted the original source code of Pool-Party.

Metrics. We evaluate C^{3}F using three metrics:

- **Success Rate**. This metric measures C^{3}F's effectiveness in successfully completing assigned AMT experiments. It is calculated as the ratio of experiments where the receiver successfully obtains any data from the channel to the total number of experiments conducted.
- **Transmission Accuracy**. This metric measures how accurately the receiver receives the intended message during covert communication. It is calculated by dividing the number of correct bits in the received sequence by the length of the original sequence. If the received sequence is longer than the original due to the system misinterpreting noise as a signal, the excess length is truncated to match the original sequence. Otherwise, if the received sequence is shorter, the missing bits are all counted as errors.
- **Bandwidth**. This metric quantifies the speed of covert communication by measuring the rate at which data is transmitted. It is calculated as the number of bits transferred per unit of time and is expressed in bits per second (bps).
- **Shannon capacity**. This metric quantifies the theoretical maximum data transmission rate of a communication channel under the presence of noise. It is calculated as the Equation (1) and is expressed in bits per second (bps).

There are two things worth noting. First, we do not consider another closely-related work Pool-Party [1] as our baseline because it does not support cross-browser or cross-mode communication. Second, we deployed the two baselines on a local device because when we tested them on AMT, the results showed none of them could successfully transmit data. There are two main reasons: first, the baselines involve complex operations (SideR) or require fine-tuning (Loophole) that exceed the capabilities of AMT workers; second, the baselines are effective only under ideal conditions but would fail in real-world scenarios where noise is prevalent. In contrast, C^{3}F is practical in all circumstances.

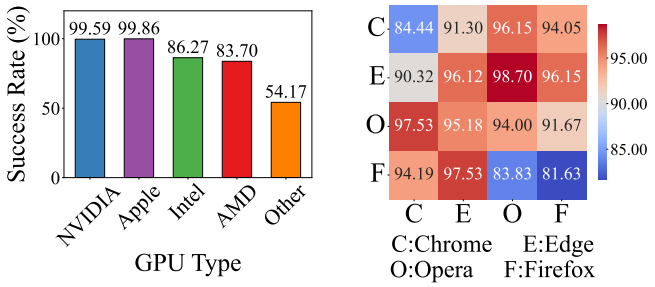
6.2. RQ1: Performance

In this RQ, we evaluate C^{3}F in real-world scenarios with prevalent noise, specifically on devices used by recruited AMT workers. The evaluation focuses on C^{3}F's performance in terms of success rate (RQ1.1), transmission accuracy (RQ1.2) and bandwidth (RQ1.3).

[RQ1.1] Success Rate. We first present a breakdown of the results from all 1,524 experiments conducted on AMT in Table 3 to determine the success rate achieved by C^{3}F. Of these data, 90 (5.90%) experiments were invalidated because the workers did not follow our instructions. Specifically, 33 experiments lacked an opened sender webpage, 29 had no active receivers, and 28 were terminated prematurely before data transmission could complete. Consequently, we consider the remaining 1,434 experiments as valid for analysis. Among them, there are 1,383/1,434 experiments that successfully established a session between the sender and receiver, with 1,331 that received identical data compared to the original message, resulting in an overall success rate of 1,331 out of 1,434 valid experiments (92.81%).

TABLE 3: [RQ1.1] A breakdown of all 1,524 experiments from the evaluation of C^{3}F on AMT. The Percentage column further shows the percentage of each subcategory over the number of valid experiments which is 1,434. In the effective experiments, the AMT workers correctly followed our instructions. Among them, "Id. Data Received" stands for identical data received, i.e., a successful transmission, and the result shows a success rate of 1,331/1,434 (92.81%).

Setup Status	Subcategory	Count	Percentage
Instructions Unfollowed (Invalid Data)	Sender Unopened	33	N/A
	Receiver Unopened	29	N/A
	Early Termination	28	N/A
Total # Invalid Experiments		90	N/A
Uncompleted	Handshake Failed	51	3.56%
Completed	Nothing Received	52	3.63%
	Id.Data Received	**1,331**	**92.81%**
Total # Valid Experiments		1,434	100%
Total		1,524	N/A



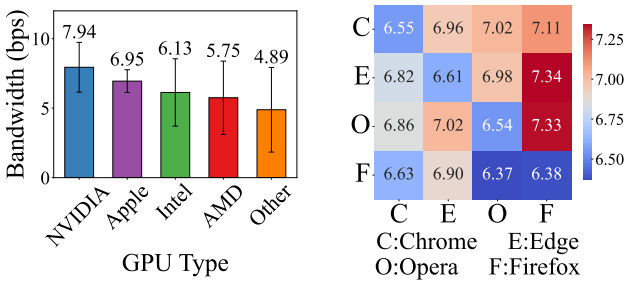
(a) Different GPUs. (b) Different Browsers.

Figure 8: [RQ1.1] (a) Success rates in AMT experiments on five different GPU types: Apple, NVIDIA, Intel, AMD and others. (b) Success rates in AMT experiments on different browser combinations. The browsers on the x-axis are senders, and those on the y-axis receivers. If the sender and receiver are using the same browser, the receiver operates in incognito mode.

We further analyze the unsuccessful transmissions: first, 51 experiments failed during the handshake in the Setup Phase, likely due to negotiation issues caused by noise. Second, 52 experiments successfully set up the session, but the receiver received no data. A potential cause for this could be that the devices of the AMT workers were overloaded with concurrent GPU rendering tasks, such as playing videos, making reliable data transmission through the GPU contention channel essentially unfeasible.

We then present the results for success rates on different GPU types in Fig. 8(a) and across different browser combinations in Fig. 8(b). The results in Fig. 8(a) indicate that: (i) C^{3}F achieves a consistently high success rate on GPUs from all major vendors, including Apple, NVIDIA, Intel and

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.



(a) Different GPUs. (b) Different Browsers.

Figure 9: [RQ1.3] (a) Average bandwidth in AMT experiments on five different GPU types. (b) Average bandwidth in AMT experiments on different browser combinations.

AMD. (ii) C^{3}F has varying success rates on different GPUs, e.g., 99.59% on NVIDIA and 83.70% on AMD. This can be attributed to architectural differences among GPUs, especially in command queue design and management, which directly impacts the performance of C^{3}F.

Next, Fig. 8(b) shows that cross-mode communication has a general lower success rate than cross-browser communication. For example, cross-mode communication on Firefox has a success rate of 81.63%, which is significantly lower than cross-browser communication between Firefox and other browsers. Besides that, The success rates across different browser combinations vary from 83.83% to 98.70%. The causes for such variation could be complex, such as different rendering engines or different jitter timers adopted to mitigate timing attacks.

[RQ1.2] Transmission Accuracy. The results show that in *all* of the 1,331 experiments where the receiver successfully received messages, the transmissions were completely accurate, achieving a 100% success rate. This high accuracy is attributed to two key components in the design of C^{3}F: the Setup Phase, where both parties negotiate an adaptive representation for bits zero and one, and the verification process using checksums.

[RQ1.3] Bandwidth. We further break down the results for bandwidth on different GPU types in Fig. 9(a) and on different browser combinations in Fig. 9(b). Similar to Fig. 8, Fig. 9 also suggests that: (i) C^{3}F maintains consistently high bandwidth on all major GPUs and bandwidth varies across different GPU architectures, as depicted in Fig. 9(a); (ii) C^{3}F for cross-browser communication has higher bandwidth compared to cross-mode communication as shown in Fig. 9(b). In addition, the types of rendering engines adopted by different browsers may also influence the bandwidth of covert communication enabled by C^{3}F.

6.3. RQ2: Comparison

In this RQ, we compare C^{3}F with two baselines, SideR [3] and Loophole [2], for cross-browser (RQ2.1) and cross-mode (RQ2.2) communication under ideal conditions with little or no noise. Note that we do not compare C^{3}F with another closely-related work Pool-Party [1] because it does not support cross-browser or cross-mode communication. Lastly, we also compare the raw channel capacities of ours and existing channels (RQ2.3).

[RQ2.1] Cross-Browser. Table 5 and Table 6 show that C^{3}F consistently outperforms SideR in all browser combinations. With Safari involved (Table 6), C^{3}F improves accuracy by 56.45% and bandwidth by 1,129.17%; without Safari (Table 5), improvements reach 99.96% on accuracy and 2,843.33% on bandwidth. Notably, C^{3}F achieves 100% accuracy across all cases, demonstrating superior practicality and robustness.

[RQ2.2] Cross-Mode. Table 6 and Table 4 show that C^{3}F consistently outperforms SideR and Loophole across all tested browsers (Chrome, Edge, Opera, Firefox, Safari). Without Safari (Table 4), C^{3}F achieves 100% accuracy—surpassing Loophole by 93.35% and SideR by 111.19%, with an average bandwidth of 8.76 bps—outperforms loophole by 75.20% and SideR by 3,404.00%. With Safari (Table 6), C^{3}F still maintains 100% accuracy, 79.86% higher than SideR, and achieves 5.88 bps bandwidth, a 2,456.52% improvement.

There are two issues worth noting. First, Loophole fails in all Safari-involved communication due to Safari's customized event loop implementation that paralyzes Loophole's design, and therefore Loophole is excluded in Table 6. Second, the lower bandwidth of C^{3}F on Safari, compared to other browsers, is mainly attributed to differences in GPU hardware: Safari runs exclusively on Macs with Apple GPUs, while the other browsers were evaluated on a Windows device with an NVIDIA GPU. This observation aligns with the trends shown in Fig. 9(a).

[RQ2.3] Shannon Capacity Comparison on Raw Channels. The purpose of this experiment is to compare the capacity of our GPU command queue contention channel with that of existing channels. The high-level idea is to separate coding schemes, e.g., those introduced by C^{3}F in Section 4, from the raw channel and directly measure and compare different contentions. Specifically, we first observe the signal and noise levels based on the signal values from the sender and the observed values from the receivers (the differences between these two are noise levels). Then, we calculate the classic Shannon-Hartley theorem formula for each channel's Shannon capacity [121] based on the observed signal and noise levels on our local Windows machine (as described in Section 3.2) based on Equation 1 below.

$$C = B \cdot \log_2\left(1 + \frac{S}{N}\right), \tag{1}$$

where B is the bandwidth, and S/N is the signal-to-noise ratio. The transmitted sequences are 512-bit random bitstrings. Each experiment was repeated ten times, resulting in a total of 40 experiments for measuring the Shannon capacity.

There are several things worth noting here. First, there are different ways of calculating the channel capacities, e.g., using the classic Shannon-Hartley theorem formula or the Blahut-Arimoto algorithm (which is used for calculating capacities of discrete channels). We choose the former because the results from the Blahut-Arimoto algorithm highly

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

TABLE 4: [RQ2.2] Cross-mode comparisons of C^3F with two baselines on transmission accuracy (Acc.) and bandwidth (Bw.).

Work	Chrome		Edge		Opera		Firefox		Any	
	Acc. (%)	Bw. (bps)	Acc. (%)	Bw. (bps)	Acc. (%)	Bw. (bps)	Acc. (%)	Bw. (bps)	Acc. (%)	Bw. (bps)
Loophole	62.17 ± 1.97	5.00 ± 0.00	34.69 ± 4.42	5.00 ± 0.00	60.41 ± 3.23	5.00 ± 0.00	49.63 ± 2.66	5.00 ± 0.00	51.72 ± 11.40	5.00 ± 0.00
SideR	52.47 ± 5.77	0.25 ± 0.00	42.90 ± 1.61	0.25 ± 0.00	43.95 ± 1.20	0.25 ± 0.00	50.07 ± 4.95	0.24 ± 0.00	47.35 ± 5.63	0.25 ± 0.00
C^3F (us)	**100.00** ± 0.00	**8.60** ± 0.07	**100.00** ± 0.00	**8.77** ± 0.15	**100.00** ± 0.00	**8.72** ± 0.15	**100.00** ± 0.00	**8.95** ± 0.08	**100.00** ± 0.00	**8.76** ± 0.17

TABLE 5: [RQ2.1] Comparison between SideR and C^3F on average transmission accuracy and average bandwidth for cross-browser communication conducted on a local Windows device.

Sender	Receiver	Accuracy (%)		Bandwidth (bps)	
		SideR	C^3F (us)	SideR	C^3F (us)
Chrome	Edge	34.44 ± 8.00	**100** ± 0.00	0.43 ± 0.09	**8.96** ± 0.09
	Opera	39.97 ± 10.32	**100** ± 0.00	0.41 ± 0.12	**8.75** ± 0.01
	Firefox	55.98 ± 8.78	**100** ± 0.00	0.25 ± 0.00	**8.87** ± 0.01
	Any	43.47 ± 12.88	**100** ± 0.00	0.37 ± 0.19	**8.86** ± 0.10
Edge	Chrome	44.14 ± 15.80	**100** ± 0.00	0.33 ± 0.12	**8.71** ± 0.28
	Opera	43.95 ± 2.50	**100** ± 0.00	0.25 ± 0.00	**8.74** ± 0.00
	Firefox	52.86 ± 4.11	**100** ± 0.00	0.25 ± 0.00	**8.83** ± 0.05
	Any	46.98 ± 10.40	**100** ± 0.00	0.28 ± 0.08	**8.76** ± 0.15
Opera	Chrome	72.98 ± 8.48	**100** ± 0.00	0.25 ± 0.00	**8.79** ± 0.05
	Edge	64.39 ± 17.73	**100** ± 0.00	0.25 ± 0.00	**8.90** ± 0.09
	Firefox	51.56 ± 2.62	**100** ± 0.00	0.21 ± 0.06	**8.75** ± 0.21
	Any	62.98 ± 14.44	**100** ± 0.00	0.24 ± 0.04	**8.81** ± 0.13
Firefox	Chrome	45.05 ± 1.99	**100** ± 0.00	0.25 ± 0.00	**9.11** ± 0.01
	Edge	45.77 ± 2.12	**100** ± 0.00	0.45 ± 0.29	**8.88** ± 0.00
	Opera	49.02 ± 0.32	**100** ± 0.00	0.24 ± 0.00	**8.72** ± 0.06
	Any	46.61 ± 2.42	**100** ± 0.00	0.31 ± 0.19	**8.90** ± 0.17
Any		50.01 ± 13.42	**100** ± 0.00	0.30 ± 0.13	**8.83** ± 0.15

TABLE 6: [RQ2.1&2.2] Comparison of average transmission accuracy and bandwidth between SideR and C^3F for Safari-involved communication—covering cross-mode (normal Safari to incognito Safari) and cross-browser (Safari to/from other browsers)—conducted on a local Mac device.

Sender	Receiver	Accuracy (%)		Bandwidth (bps)	
		SideR	C^3F (us)	SideR	C^3F (us)
Safari	Safari	55.60 ± 3.93	**100** ± 0.00	0.23 ± 0.00	**5.88** ± 0.06
Safari	Chrome	60.42 ± 12.97	**100** ± 0.00	0.63 ± 0.42	**5.92** ± 0.09
	Edge	66.80 ± 13.44	**100** ± 0.00	0.69 ± 0.39	**5.88** ± 0.09
	Opera	67.45 ± 14.69	**100** ± 0.00	1.53 ± 1.05	**5.94** ± 0.23
	Firefox	77.08 ± 1.13	**100** ± 0.00	0.24 ± 0.00	5.90 ± 0.01
	Any	65.47 ± 11.81	**100** ± 0.00	0.66 ± 0.67	**5.91** ± 0.10
Chrome	Safari	63.28 ± 2.38	**100** ± 0.00	0.26 ± 0.01	**6.07** ± 0.14
Edge		61.20 ± 3.03	**100** ± 0.00	0.26 ± 0.01	**5.84** ± 0.07
Firefox		58.85 ± 2.77	**100** ± 0.00	0.24 ± 0.00	**5.70** ± 0.10
Opera		64.58 ± 0.81	**100** ± 0.00	0.26 ± 0.01	**5.92** ± 0.04
Any		60.70 ± 4.07	**100** ± 0.00	0.25 ± 0.01	**5.88** ± 0.14
Any		63.92 ± 9.07	**100** ± 0.00	0.48 ± 0.53	**5.90** ± 0.13

depend on the digitalization method, e.g., encoding one or two or more bits in one signal based on the threshold, and we want to measure the capacity without any coding schemes. Second, the obtained Shannon capacity highly depends on the noise level, which will vary if the noise level increases. Our experiment is performed on a local machine, where the noise level is very small. In a real-world environment, the noise could be much larger, and that is also why none of the existing channels work in an Amazon Turk environment.

TABLE 7: [RQ2.3] Comparison of the average Shannon capacity of different raw channels exploited by Pool-Party, Loophole, SideR, and C^3F, evaluated on Chrome running on a local Windows device.

Raw Channel Used by	C^3F	Pool-Party	Loophole	SideR
Capacity (bps)	**53.66** ± 0.52	18.07 ± 0.00	1.69 ± 0.13	0.01 ± 0.00

Table 7 shows the results, that is, Shannon capacities, of the raw channels used in all works. Our GPU command queue contention channel clearly outperforms existing raw channels. Since the capacity is an upper bound, the capacity of C^3F is significantly larger than what can be achieved in practice for real-world, lossless communications.

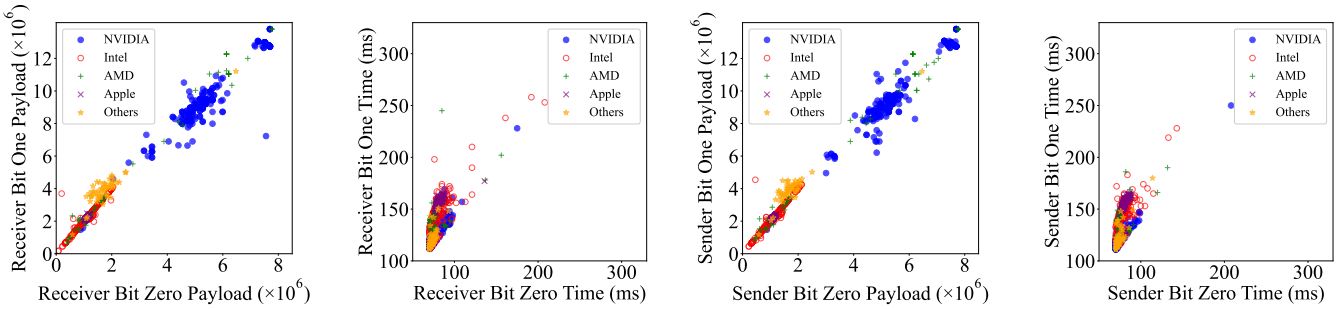
6.4. RQ3: Intermediate Results

In this RQ, we present the intermediate results obtained during the experiments on the AMT workers' devices, including task payload determination (see Section 4.2.1) and retransmission (see Section 4.2.2). In addition, we also provide evaluation results for GPU utilization of C^3F.

[RQ3.1] Task Payload Analysis. We first present data for the task payloads computed by C^3F for both communicating parties and the corresponding rendering time in Fig. 10. From Fig. 10 (a) and Fig. 10 (c), we observe a clear linear relationship between the payloads for bits zero and one, which is consistent with the algorithm described in Section 4.2.1. In addition, C^3F compels high-performance GPUs to handle more task payloads while maintaining rendering time similar to other GPUs, resulting in a cluster in the upper right corner of Fig. 10 (a) and Fig. 10 (c). Meanwhile, across all tested GPUs, C^3F determines a consistent level of rendering time for both bit zero and one, forming a cluster in the lower left of Fig. 10 (b) and Fig. 10 (d). The results demonstrate the flexibility of C^3F across GPUs with varying performance levels.

[RQ3.2] Retransmission Analysis. Next, we present results obtained from retransmission analysis. First, a total of 645 retransmissions were recorded across all AMT experiments with received data (1,331), with an average of 0.49 retransmissions per experiment. Second, we examine how retransmissions impact bandwidth during the AMT experiment, as illustrated in Fig. 11. The results reveal a clear trend for 1,193/1,331 (89.63%) experiments: the number of retransmissions is inversely related to bandwidth. This relationship is reasonable, as an increase in retransmissions extends the total time for transmission, and thus reducing

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.



(a) Receiver Payload (b) Receiver Time (c) Sender Payload (d) Sender Time

Figure 10: [RQ3.1] Task Payloads (i.e., for loop iterations of a single operation) determined by C^{3}F and rendering time measured by C^{3}F to represent bits zero and one for both the sender and receiver on different GPUs in the AMT experiments.

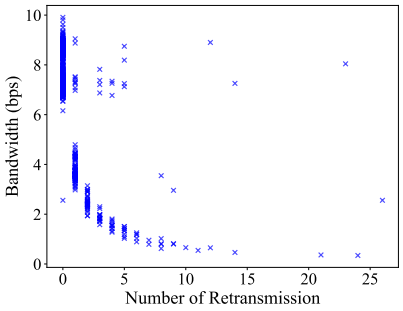


Figure 11: [RQ3.2] Impact of retransmissions on bandwidth in the AMT experiment.

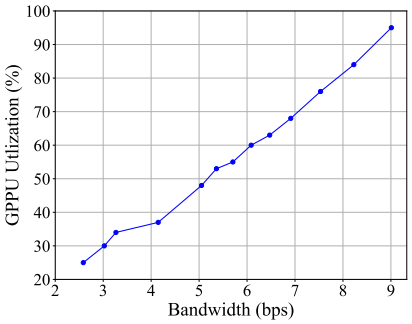


Figure 12: [RQ3.3] GPU utilization of C^{3}F.

bandwidth. Nonetheless, there are anomalies where a high number of retransmissions coincides with high bandwidth. These anomalies could be attributed to high-performance GPUs, which empowers C^{3}F to maintain a high level of bandwidth even when retransmissions occur.

[RQ3.3] GPU Utilization. In this RQ, we evaluate the GPU utilization of C^{3}F according to different bandwidths. Specifically, we change the frequency of payloads on the sender so that the channel's bandwidth changes accordingly. The receiver still continuously sends probes because probes are lightweight and do not consume a lot of GPU resources. The evaluation was conducted on the same device described in Section 3.2. We then present the results in Fig. 12, showing C^{3}F's median GPU utilization based on measured bandwidth. The GPU utilization increases to almost 100% for a maximized bandwidth. At the same time, GPU utilization can be reduced when the bandwidth is lower. For example, a 6 bps bandwidth maps to about 60% GPU utilization.

TABLE 8: [RQ4] Tor browser as a defense: performance degradation of C^{3}F on cross-mode and cross-browser communication when the Tor browser is involved. Note that when the sender and the receiver are both Tor, the receiver is in incognito.

Sender	Receiver	Success Rate (%)	Retrans. Rate (%)	Bandwidth (bps)
Tor	Tor	100 ± 0.00	17.97 ± 25.41	4.68 ± 0.15
Tor	Chrome	100 ± 0.00	0 ± 0.00	5.53 ± 0.13
	Edge	100 ± 0.00	0 ± 0.00	5.67 ± 0.14
	Opera	100 ± 0.00	0 ± 0.00	5.51 ± 0.18
	Firefox	100 ± 0.00	0 ± 0.00	5.63 ± 0.05
Chrome	Tor	100 ± 0.00	0.36 ± 0.37	5.56 ± 0.10
Edge		100 ± 0.00	0 ± 0.00	5.41 ± 0.04
Opera		100 ± 0.00	16.15 ± 22.83	4.68 ± 0.08
Firefox		100 ± 0.00	17.45 ± 24.68	4.71 ± 0.02
Any		100 ± 0.00	4.42 ± 14.20	5.30 ± 0.14

TABLE 9: [RQ4] Analysis of a fuzzy-time-based defense against C^{3}F that we deployed on the Chrome browser.

Timer Level	Success Rate	Accuracy	Bandwidth (bps)
0 ms (No defense)	100%	100%	8.60 ± 0.07
5 ms	100%	100%	8.64 ± 0.21
10 ms	100%	100%	7.98 ± 1.56
15 ms	100%	100%	6.45 ± 2.22

6.5. RQ4: Defense Analysis

In this RQ, we study whether existing defense mechanisms can mitigate covert communication enabled by C^{3}F.

Many works [4], [105]–[108] have proposed adding random delays as a mitigation strategy for timing-based attacks. To evaluate the impact of this strategy on C^{3}F, first, we selected the Tor browser due to its use of jitters to mitigate timing channels [105], and then deployed C^{3}F on Tor browser to test the performance degradation for both cross-mode and cross-browser communication. Each browser combination was tested ten times on the same Windows device described in Section 3.2.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

The results shown in Table 8 indicate that the defense partially mitigates C^{3}F. Specifically, C^{3}F achieves success rates of 100% in all cases, but the retransmission rate increases for many browser pairs, and therefore the bandwidth drops as well. Interestingly, when Tor is the sender, we do not observe any retransmission. When Tor is the receiver, retransmission is very often due to Tor's introduced noise. The reason is that the receiver needs a more accurate measurement of time due to the probing operation as compared with the sender.

Second, besides the evaluation on Tor browser, we further introduced random delays of varying levels to the results of `Date.now()` and used cross-mode communication in the Chrome browser to evaluate C^{3}F's performance, including success rates, transmission accuracy, and bandwidth. Each delay level was tested ten times on the same Windows device.

As shown in Table 9, a defense level of 5 ms has minimal impact on C^{3}F, except for a slightly higher standard deviation. At delay levels of 10 ms and 15 ms, both the success rate and accuracy remain at 100%, as the handshake stage of C^{3}F is robust against the introduced delays. Meanwhile, the bandwidth experiences a reduction along with a larger standard deviation, since the added delay forces C^{3}F to allocate longer rendering time to distinguish between bits zero and one.

7. Discussion

GPU Command Queue as a Side Channel. Contention in the GPU command queue could also be exploited as a side channel. Specifically, the contention between graphics rendering tasks and general-purpose computation tasks may reveal sensitive information such as the rendering process of webpages, the inference process of AI tasks, and the execution of cryptographic algorithms. We leave the side channel attack, an out-of-scope topic, for future work.

Other Defense Mechanisms. Cao et al. [110] propose a deterministic timer that generates fixed intervals between consecutive frames. Reis et al. [116] propose site isolation that is effective against browser-level side channels. Finally, partitioning GPU resources for each browser instance could theoretically mitigate C^{3}F , but this defense raises a major problem of ensuring efficient GPU utilization. If resources are statically allocated, each instance receives a fixed share: allocations that are too small slow down computation, while allocations that are too large lead to wasted capacity. Dynamic allocation could address this mismatch, but accurately predicting the resource requirements of each task remains a significant challenge. We leave a thorough analysis for these defense approaches for future work.

Potential Generalization of the Framework. While the layer-based C^{3}F framework can theoretically be adapted to other cross-mode or cross-browser covert channels by modifying the Raw Channel and Session Layers to fit the characteristics of a given channel, the performance of C^{3}F degrades significantly on noisy channels. In particular, channels exploited by Loophole and SideR suffer from high noise, resulting in frequent bit errors and retransmissions that make C^{3}F ineffective. We leave generalization of the C^{3}F framework for future work.

Ethics. We discuss two aspects: (i) human subject research and (ii) responsible disclosure. First, we applied for our Institutional Review Board (IRB) approval before the experiments and obtained an "exempt" decision. Specifically, our AMT experiments did not collect any Personal Identifiable Information (PII), e.g., IP addresses. In addition, all transmitted data are generated randomly with no real meaning. Our experiment does collect GPU information and browser types, which are non-identifiable information. Participants are made aware of such data collection and given the option of opt-out. Second, we also responsibly disclosed our findings to affected browser vendors and gave them 45 days. The detailed disclosure timeline is provided in the Appendix. So far, Firefox and Tor vendors have confirmed our attack and are working with us on practical defenses.

8. Conclusion

In this paper, we discover a novel covert channel that leverages GPU command queue contention, and then design and implement C^{3}F, a framework that enables reliable cross-mode and cross-browser covert communication based on the discovered channel. Our key insight is that OpenGL synchronization can make a receiver's probe task wait for the sender's payload task, thus forming a contention. Then, our framework C^{3}F negotiates a proper threshold for the sender's payload through a handshake stage and then facilitates the communication with error correction and retransmissions. Our evaluation with AMT workers showed that C^{3}F achieves 100% accuracy once transmission is complete. Local evaluations further demonstrate that C^{3}F consistently outperforms baselines under all tested circumstances. We have responsibly disclosed our findings to browser vendors, open-sourced a prototype [8] of C^{3}F, and provided a live demo on an anonymous website [9] for demonstration.

Acknowledgments

We would like to thank anonymous shepherd and reviewers for their helpful comments and feedback. This work was supported by the National Key R&D Program of China under grants 2024YFE0203800 and the National Natural Science Foundation of China under Grant No. 62302438. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the National Key R&D Program of China and the National Natural Science Foundation of China.

References

[1] P. Snyder, S. Karami, A. Edelstein, B. Livshits, and H. Haddadi, "Pool-Party: Exploiting browser resource pools for web tracking,"

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 7091–7105. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/snyder

[2] P. Vila and B. Kopf, "Loophole: Timing attacks on shared event loops in chrome," in *26th USENIX Security Symposium (USENIX Security 17)*. Vancouver, BC: USENIX Association, Aug. 2017, pp. 849–864. [Online]. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/vila

[3] S. Wu, J. Yu, M. Yang, and Y. Cao, "Rendering contention channel made practical in web browsers," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 3183–3199. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/wu-shujiang

[4] Y. Oren, V. P. Kemerlis, S. Sethumadhavan, and A. D. Keromytis, "The spy in the sandbox: Practical cache attacks in javascript and their implications," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 1406–1418.

[5] D. Genkin, L. Pachmanov, E. Tromer, and Y. Yarom, "Drive-by key-extraction cache attacks from portable code," in *Applied Cryptography and Network Security: 16th International Conference, ACNS 2018, Leuven, Belgium, July 2-4, 2018, Proceedings 16*. Springer, 2018, pp. 83–102.

[6] T. Rokicki, C. Maurice, M. Botvinnik, and Y. Oren, "Port contention goes portable: Port contention side channels in web browsers," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, 2022, pp. 1182–1194.

[7] [opengl wiki] synchronization. https://www.khronos.org/opengl/wiki/Synchronization.

[8] Jinhong Liu, "Repository: C3f," https://github.com/HugoLeoLJH/C3F, accessed: 2025-10-10.

[9] ——, "C3f demo: Covert channel framework," https://hugoleoljh.github.io/c3f-demo/c3f/receiver.html, 2025, accessed: 2025-10-10.

[10] Amazon Mechanical Turk, "Amazon mechanical turk," https://www.mturk.com/, accessed: 2024-08-31.

[11] P. Calvano. (2020) An analysis of cookie sizes on the web. https://paulcalvano.com/2020-07-13-an-analysis-of-cookie-sizes-on-the-web/.

[12] Average time on page - definition, formula & tips. https://agencyanalytics.com/kpi-definitions/average-time-on-page.

[13] C. Maurice, M. Weber, M. Schwarz, L. Giner, D. Gruss, C. A. Boano, S. Mangard, and K. Römer, "Hello from the other side: Ssh over robust cache covert channels in the cloud." in *NDSS*, vol. 17, 2017, pp. 8–11.

[14] H. Naghibijouybari, K. N. Khasawneh, and N. Abu-Ghazaleh, "Constructing and characterizing covert channels on gpgpus," in *Proceedings of the 50th annual IEEE/ACM international symposium on microarchitecture*, 2017, pp. 354–366.

[15] G. Acar, C. Eubank, S. Englehardt, M. Juarez, A. Narayanan, and C. Diaz, "The web never forgets: Persistent tracking mechanisms in the wild," in *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*, 2014, pp. 674–689.

[16] F. Alaca and P. C. Van Oorschot, "Device fingerprinting for augmenting web authentication: classification and analysis of methods," in *Proceedings of the 32nd annual conference on computer security applications*, 2016, pp. 289–301.

[17] Y. Cao, S. Li, and E. Wijmans, "(cross-) browser fingerprinting via os and hardware level features," in *Proceedings 2017 Network and Distributed System Security Symposium*. Internet Society, 2017.

[18] P. Laperdrix, W. Rudametkin, and B. Baudry, "Beauty and the beast: Diverting modern web browsers to build unique browser fingerprints," in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 878–894.

[19] R. Upathilake, Y. Li, and A. Matrawy, "A classification of web browser fingerprinting techniques," in *2015 7th International Conference on New Technologies, Mobility and Security (NTMS)*. IEEE, 2015, pp. 1–5.

[20] S. Wu, S. Li, Y. Cao, and N. Wang, "Rendered private: Making {GLSL} execution uniform to prevent {WebGL-based} browser fingerprinting," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1645–1660.

[21] P. Laperdrix, N. Bielova, B. Baudry, and G. Avoine, "Browser fingerprinting: A survey," *ACM Transactions on the Web (TWEB)*, vol. 14, no. 2, pp. 1–33, 2020.

[22] S. Li and Y. Cao, "Who touched my browser fingerprint? a large-scale measurement study and classification of fingerprint dynamics," in *Proceedings of the ACM Internet Measurement Conference*, 2020, pp. 370–385.

[23] N. Andriamilanto, T. Allard, and G. L. Guelvouit, ""guess who?" large-scale data-centric study of the adequacy of browser fingerprints for web authentication," in *Innovative Mobile and Internet Services in Ubiquitous Computing: Proceedings of the 14th International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing (IMIS-2020)*. Springer, 2021, pp. 161–172.

[24] I. Sanchez-Rola, I. Santos, and D. Balzarotti, "Clock around the clock: Time-based device fingerprinting," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1502–1514.

[25] S. Wu, P. Sun, Y. Zhao, and Y. Cao, "Him of many faces: Characterizing billion-scale adversarial and benign browser fingerprints on commercial websites." in *NDSS*, 2023.

[26] A. Shusterman, A. Agarwal, S. O'Connell, D. Genkin, Y. Oren, and Y. Yarom, "{Prime+ Probe} 1,{JavaScript} 0: Overcoming browser-based {Side-Channel} defenses," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2863–2880.

[27] A. Shusterman, Z. Avraham, E. Croitoru, Y. Haskal, L. Kang, D. Levi, Y. Meltser, P. Mittal, Y. Oren, and Y. Yarom, "Website fingerprinting through the cache occupancy channel and its real world practicality," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2042–2060, 2020.

[28] H. Kim, S. Lee, and J. Kim, "Inferring browser activity and status through remote monitoring of storage usage," in *Proceedings of the 32nd Annual Conference on Computer Security Applications*, 2016, pp. 410–421.

[29] E. Ferguson, A. Wilson, and H. Naghibijouybari, "Webgpu-spy: Finding fingerprints in the sandbox through gpu cache attacks," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, 2024, pp. 158–171.

[30] J. Booth, "Not so incognito: Exploiting resource-based side channels in javascript engines," Ph.D. dissertation, 2015.

[31] H. Naghibijouybari, A. Neupane, Z. Qian, and N. Abu-Ghazaleh, "Rendered insecure: Gpu side channel attacks are practical," in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 2139–2153.

[32] N. Matyunin, Y. Wang, T. Arul, K. Kullmann, J. Szefer, and S. Katzenbeisser, "Magneticspy: Exploiting magnetometer in mobile devices for website and application fingerprinting," in *Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society*, 2019, pp. 135–149.

[33] M. Lipp, D. Gruss, M. Schwarz, D. Bidner, C. Maurice, and S. Mangard, "Practical keystroke timing attacks in sandboxed javascript," in *Computer Security–ESORICS 2017: 22nd European Symposium on Research in Computer Security, Oslo, Norway, September 11-15, 2017, Proceedings, Part II 22*. Springer, 2017, pp. 191–209.

[34] D. Wang, A. Neupane, Z. Qian, N. B. Abu-Ghazaleh, S. V. Krishnamurthy, E. J. Colbert, and L. Y. Paul, "Unveiling your keystrokes: A cache-based side-channel attack on graphics libraries." in *NDSS*, 2019.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

[35] P. Pessl, D. Gruss, C. Maurice, M. Schwarz, and S. Mangard, "{DRAMA}: Exploiting {DRAM} addressing for {Cross-CPU} attacks," in *25th USENIX security symposium (USENIX security 16)*, 2016, pp. 565–581.

[36] A. Bortz and D. Boneh, "Exposing private information by timing web applications," in *Proceedings of the 16th international conference on World Wide Web*, 2007, pp. 621–628.

[37] Q. Ge, Y. Yarom, D. Cock, and G. Heiser, "A survey of microarchitectural timing attacks and countermeasures on contemporary hardware," *Journal of Cryptographic Engineering*, vol. 8, pp. 1–27, 2018.

[38] P. Stone, "Pixel perfect timing attacks with html5," *Context Information Security (White Paper)*, 2013.

[39] M. Smith, C. Disselkoen, S. Narayan, F. Brown, and D. Stefan, "Browser history {re: visited}," in *12th USENIX Workshop on Offensive Technologies (WOOT 18)*, 2018.

[40] A. Huang, C. Zhu, D. Wu, Y. Xie, and X. Luo, "An adaptive method for cross-platform browser history sniffing," in *Measurements, Attacks, and Defenses for the Web Workshop*, 2020, pp. 1–7.

[41] Z. Weinberg, E. Y. Chen, P. R. Jayaraman, and C. Jackson, "I still know what you visited last summer: Leaking browsing history via user interaction and side channel attacks," in *2011 IEEE Symposium on Security and Privacy*. IEEE, 2011, pp. 147–161.

[42] R. Kotcher, Y. Pei, P. Jumde, and C. Jackson, "Cross-origin pixel stealing: timing attacks using css filters," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 1055–1062.

[43] T. Van Goethem, W. Joosen, and N. Nikiforakis, "The clock is still ticking: Timing attacks in the modern web," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 1382–1393.

[44] T. Van Goethem, M. Vanhoef, F. Piessens, and W. Joosen, "Request and conquer: Exposing cross-origin resource size," in *25th {USENIX} Security Symposium ({USENIX} Security 16)*, 2016, pp. 447–462.

[45] T. Rokicki, C. Maurice, and M. Schwarz, "Cpu port contention without smt," in *European Symposium on Research in Computer Security*. Springer, 2022, pp. 209–228.

[46] D. A. Osvik, A. Shamir, and E. Tromer, "Cache attacks and countermeasures: the case of aes," in *Topics in Cryptology–CT-RSA 2006: The Cryptographers' Track at the RSA Conference 2006, San Jose, CA, USA, February 13-17, 2005. Proceedings*. Springer, 2006, pp. 1–20.

[47] K. Okamura and Y. Oyama, "Load-based covert channels between xen virtual machines," in *Proceedings of the 2010 ACM Symposium on Applied Computing*, 2010, pp. 173–180.

[48] C. Su and Q. Zeng, "Survey of cpu cache-based side-channel attacks: Systematic analysis, security models, and countermeasures," *Security and Communication Networks*, vol. 2021, no. 1, p. 5559552, 2021.

[49] N. Matyunin, N. A. Anagnostopoulos, S. Boukoros, M. Heinrich, A. Schaller, M. Kolinichenko, and S. Katzenbeisser, "Tracking private browsing sessions using cpu-based covert channels," in *Proceedings of the 11th ACM Conference on Security & Privacy in Wireless and Mobile Networks*, 2018, pp. 63–74.

[50] S. Jana and V. Shmatikov, "Memento: Learning secrets from process footprints," in *2012 IEEE Symposium on Security and Privacy*. IEEE, 2012, pp. 143–157.

[51] M. Guri, "Magneto: Covert channel between air-gapped systems and nearby smartphones via cpu-generated magnetic fields," *Future Generation Computer Systems*, vol. 115, pp. 115–125, 2021.

[52] Y. Yarom and K. Falkner, "{FLUSH+ RELOAD}: A high resolution, low noise, l3 cache {Side-Channel} attack," in *23rd USENIX security symposium (USENIX security 14)*, 2014, pp. 719–732.

[53] M. Lipp, D. Gruss, R. Spreitzer, C. Maurice, and S. Mangard, "{ARMageddon}: Cache attacks on mobile devices," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 549–564.

[54] L. Giner, R. Czerny, C. Gruber, F. Rauscher, A. Kogler, D. D. A. Braga, and D. Gruss, "Generic and automated drive-by gpu cache attacks from the browser," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, 2024, pp. 128–140.

[55] H. Naghibijouybari, E. M. Koruyeh, and N. Abu-Ghazaleh, "Microarchitectural attacks in heterogeneous systems: A survey," *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–40, 2022.

[56] Z. H. Jiang, Y. Fei, and D. Kaeli, "A complete key recovery timing attack on a gpu," in *2016 IEEE International symposium on high performance computer architecture (HPCA)*. IEEE, 2016, pp. 394–405.

[57] ——, "A novel side-channel timing attack on gpus," in *Proceedings of the on Great Lakes Symposium on VLSI 2017*, 2017, pp. 167–172.

[58] ——, "Exploiting bank conflict-based side-channel timing leakage of gpus," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 16, no. 4, pp. 1–24, 2019.

[59] J. Ahn, C. Jin, J. Kim, M. Rhu, Y. Fei, D. Kaeli, and J. Kim, "Trident: A hybrid correlation-collision gpu cache timing attack for aes key recovery," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 332–344.

[60] H. Taneja, J. Kim, J. J. Xu, S. Van Schaik, D. Genkin, and Y. Yarom, "Hot pixels: Frequency, power, and temperature attacks on {GPUs} and arm {SoCs}," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 6275–6292.

[61] P. Cronin, X. Gao, H. Wang, and C. Cotton, "An exploration of arm system-level cache and gpu side channels," in *Proceedings of the 37th Annual Computer Security Applications Conference*, 2021, pp. 784–795.

[62] M. Guri, "Gpu-fan: Leaking sensitive data from air-gapped machines via covert noise from gpu fans," in *Nordic Conference on Secure IT Systems*. Springer, 2022, pp. 194–211.

[63] M. S. Inci, B. Gulmezoglu, G. Irazoqui, T. Eisenbarth, and B. Sunar, "Cache attacks enable bulk key recovery on the cloud," in *Cryptographic Hardware and Embedded Systems–CHES 2016: 18th International Conference, Santa Barbara, CA, USA, August 17-19, 2016, Proceedings 18*. Springer, 2016, pp. 368–388.

[64] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, "Last-level cache side-channel attacks are practical," in *2015 IEEE symposium on security and privacy*. IEEE, 2015, pp. 605–622.

[65] Y. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Cross-vm side channels and their use to extract private keys," in *Proceedings of the 2012 ACM conference on Computer and communications security*, 2012, pp. 305–316.

[66] D. Gruss, C. Maurice, K. Wagner, and S. Mangard, "Flush+ flush: a fast and stealthy cache attack," in *Detection of Intrusions and Malware, and Vulnerability Assessment: 13th International Conference, DIMVA 2016, San Sebastián, Spain, July 7-8, 2016, Proceedings 13*. Springer, 2016, pp. 279–299.

[67] G. Saileshwar, C. W. Fletcher, and M. Qureshi, "Streamline: a fast, flushless cache covert-channel attack by enabling asynchronous collusion," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 1077–1090.

[68] D. Gruss, R. Spreitzer, and S. Mangard, "Cache template attacks: Automating attacks on inclusive {Last-Level} caches," in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 897–912.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

[69] Q. Jiang and C. Wang, "{Sync+ Sync}: A covert channel built on fsync with storage," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 3349–3366.

[70] Q. Xiao, M. K. Reiter, and Y. Zhang, "Mitigating storage side channels using statistical privacy mechanisms," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 1582–1594.

[71] F. Armknecht, C. Boyd, G. T. Davies, K. Gjøsteen, and M. Toorani, "Side channels in deduplication: Trade-offs between leakage and efficiency," in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, 2017, pp. 266–274.

[72] M. Guri, "Etherled: sending covert morse signals from air-gapped devices via network card (nic) leds," in *2022 IEEE International Conference on Cyber Security and Resilience (CSR)*. IEEE, 2022, pp. 163–170.

[73] M. Guri, B. Zadov, and Y. Elovici, "Led-it-go: Leaking (a lot of) data from air-gapped computers via the (small) hard drive led," in *Detection of Intrusions and Malware, and Vulnerability Assessment: 14th International Conference, DIMVA 2017, Bonn, Germany, July 6-7, 2017, Proceedings 14*. Springer, 2017, pp. 161–184.

[74] M. Guri, B. Zadov, D. Bykhovsky, and Y. Elovici, "Ctrl-alt-led: Leaking data from air-gapped computers via keyboard leds," in *2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*, vol. 1. IEEE, 2019, pp. 801–810.

[75] M. Guri, "Optical covert channel from air-gapped networks via remote orchestration of router/switch leds," in *2018 European Intelligence and Security Informatics Conference (EISIC)*. IEEE, 2018, pp. 54–60.

[76] ——, "Gairoscope: Leaking data from air-gapped computers to nearby smartphones using speakers-to-gyro communication," in *2021 18th International Conference on Privacy, Security and Trust (PST)*. IEEE, 2021, pp. 1–10.

[77] ——, "Power-supplay: Leaking sensitive data from air-gapped, audio-gapped systems by turning the power supplies into speakers," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 1, pp. 313–330, 2021.

[78] M. Guri, Y. Solewicz, and Y. Elovici, "Mosquito: Covert ultrasonic transmissions between two air-gapped computers using speaker-to-speaker communication," in *2018 IEEE Conference on Dependable and Secure Computing (DSC)*. IEEE, 2018, pp. 1–8.

[79] M. Guri, Y. Solewicz, A. Daidakulov, and Y. Elovici, "Acoustic data exfiltration from speakerless air-gapped computers via covert hard-drive noise ('diskfiltration')," in *Computer Security–ESORICS 2017: 22nd European Symposium on Research in Computer Security, Oslo, Norway, September 11-15, 2017, Proceedings, Part II 22*. Springer, 2017, pp. 98–115.

[80] R. Hund, C. Willems, and T. Holz, "Practical timing side channel attacks against kernel space aslr," in *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, pp. 191–205.

[81] N. Matyunin, J. Szefer, S. Biedermann, and S. Katzenbeisser, "Covert channels using mobile device's magnetic field sensors," in *2016 21st Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2016, pp. 525–532.

[82] F. Brasser, U. Müller, A. Dmitrienko, K. Kostiainen, S. Capkun, and A.-R. Sadeghi, "Software grand exposure:{SGX} cache attacks are practical," in *11th USENIX workshop on offensive technologies (WOOT 17)*, 2017.

[83] F. Dall, G. De Micheli, T. Eisenbarth, D. Genkin, N. Heninger, A. Moghimi, and Y. Yarom, "Cachequote: Efficiently recovering long-term secrets of sgx epid via cache attacks," 2018.

[84] S. Lee, M.-W. Shih, P. Gera, T. Kim, H. Kim, and M. Peinado, "Inferring fine-grained control flow inside {SGX} enclaves with branch shadowing," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 557–574.

[85] P. Chen, L. Li, and Z. Yang, "{Cross-VM} and {Cross-Processor} covert channels exploiting processor idle power management," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 733–750.

[86] Y. Zhang, C. Slocum, J. Chen, and N. Abu-Ghazaleh, "It's all in your head (set): Side-channel attacks on {AR/VR} systems," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 3979–3996.

[87] M. Guri, M. Monitz, and Y. Elovici, "Usbee: Air-gap covert-channel via electromagnetic emission from usb," in *2016 14th Annual Conference on Privacy, Security and Trust (PST)*. IEEE, 2016, pp. 264–268.

[88] M. Guri, "Cd-leak: Leaking secrets from audioless air-gapped computers using covert acoustic signals from cd/dvd drives," in *2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2020, pp. 808–816.

[89] M. Guri, A. Kachlon, O. Hasson, G. Kedma, Y. Mirsky, and Y. Elovici, "{GSMem}: Data exfiltration from {Air-Gapped} computers over {GSM} frequencies," in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 849–864.

[90] M. Guri, G. Kedma, A. Kachlon, and Y. Elovici, "Airhopper: Bridging the air-gap between isolated networks and mobile phones using radio frequencies," in *2014 9th International Conference on Malicious and Unwanted Software: The Americas (MALWARE)*. IEEE, 2014, pp. 58–67.

[91] M. Guri, B. Zadov, D. Bykhovsky, and Y. Elovici, "Powerhammer: Exfiltrating data from air-gapped computers through power lines," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 1879–1890, 2019.

[92] M. Guri, B. Zadov, A. Daidakulov, and Y. Elovici, "xled: Covert data exfiltration from air-gapped networks via switch and router leds," in *2018 16th Annual Conference on Privacy, Security and Trust (PST)*. IEEE, 2018, pp. 1–12.

[93] M. Guri, D. Bykhovsky, and Y. Elovici, "Brightness: Leaking sensitive data from air-gapped workstations via screen brightness," in *2019 12th CMI Conference on Cybersecurity and Privacy (CMI)*. IEEE, 2019, pp. 1–6.

[94] M. Guri, B. Zadov, and Y. Elovici, "Odini: Escaping sensitive data from faraday-caged, air-gapped computers via magnetic fields," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 1190–1203, 2019.

[95] M. Guri, Y. Solewicz, and Y. Elovici, "Fansmitter: Acoustic data exfiltration from air-gapped computers via fans noise," *Computers & Security*, vol. 91, p. 101721, 2020.

[96] M. Guri, O. Hasson, G. Kedma, and Y. Elovici, "An optical covert-channel to leak data through an air-gap," in *2016 14th annual conference on privacy, security and trust (pst)*. IEEE, 2016, pp. 642–649.

[97] M. Guri, "Pixhell attack: Leaking sensitive information from air-gap computers via 'singing pixels'," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 976–987.

[98] ——, "Air-gap electromagnetic covert channel," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 2127–2144, 2023.

[99] ——, "Satan: Air-gap exfiltration attack via radio signals from sata cables," in *2022 19th Annual International Conference on Privacy, Security & Trust (PST)*. IEEE, 2022, pp. 1–10.

[100] ——, "Air-fi: Leaking data from air-gapped computers using wi-fi frequencies," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 3, pp. 2547–2564, 2022.

[101] ——, "Lantenna: Exfiltrating data from air-gapped networks via ethernet cables emission," in *2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2021, pp. 745–754.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

[102] ——, "Exfiltrating data from air-gapped computers via vibrations," *Future Generation Computer Systems*, vol. 122, pp. 69–81, 2021.

[103] ——, "Hotspot: Crossing the air-gap between isolated pcs and nearby smartphones using temperature," in *2019 European Intelligence and Security Informatics Conference (EISIC)*. IEEE, 2019, pp. 94–100.

[104] ——, "Rambo: Leaking secrets from air-gap computers by spelling covert radio signals from computer ram," in *Nordic Conference on Secure IT Systems*. Springer, 2023, pp. 144–161.

[105] D. Kohlbrenner and H. Shacham, "Trusted browsers for uncertain times," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 463–480.

[106] B. Zbarsky, "Clamp the resolution of performance.now() calls to 5us," https://hg.mozilla.org/integration/mozilla-inbound/rev/48ae8b5e62ab, 2015, accessed: 2024-08-31.

[107] Chromium Project, "window.performance.now does not support sub-millisecond precision on windows," https://bugs.chromium.org/p/chromium/issues/detail?id=158234##c110, 2016, accessed: 2024-08-31.

[108] B. C. Vattikonda, S. Das, and H. Shacham, "Eliminating fine grained timers in xen," in *Proceedings of the 3rd ACM workshop on Cloud computing security workshop*, 2011, pp. 41–46.

[109] W. Wu and B. Ford, "Deterministically deterring timing attacks in deterland," *arXiv preprint arXiv:1504.07070*, 2015.

[110] Y. Cao, Z. Chen, S. Li, and S. Wu, "Deterministic browser," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. ACM, 2017, pp. 163–178. [Online]. Available: https://doi.org/10.1145/3133956.3133996

[111] L. Domnitser, A. Jaleel, J. Loew, N. Abu-Ghazaleh, and D. Ponomarev, "Non-monopolizable caches: Low-complexity mitigation of cache side channel attacks," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 8, no. 4, pp. 1–21, 2012.

[112] Q. Ge, Y. Yarom, T. Chothia, and G. Heiser, "Time protection: the missing os abstraction," in *Proceedings of the Fourteenth EuroSys Conference 2019*, 2019, pp. 1–17.

[113] R. Sprabery, K. Evchenko, A. Raj, R. B. Bobba, S. Mohan, and R. H. Campbell, "A novel scheduling framework leveraging hardware cache partitioning for cache-side-channel elimination in clouds," *arXiv preprint arXiv:1708.09538*, 2017.

[114] Q. Xu, H. Naghibijouybari, S. Wang, N. Abu-Ghazaleh, and M. Annavaram, "Gpuguard: Mitigating contention based side and covert channel attacks on gpus," in *Proceedings of the ACM International Conference on Supercomputing*, 2019, pp. 497–509.

[115] M. Schwarz, C. Maurice, D. Gruss, and S. Mangard, "Fantastic timers and where to find them: High-resolution microarchitectural attacks in javascript," in *Financial Cryptography and Data Security: 21st International Conference, FC 2017, Sliema, Malta, April 3-7, 2017, Revised Selected Papers 21*. Springer, 2017, pp. 247–267.

[116] C. Reis, A. Moshchuk, and N. Oskov, "Site isolation: Process separation for web sites within the browser," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1661–1678.

[117] T. Kim, M. Peinado, and G. Mainar-Ruiz, "{STEALTHMEM}:{System-Level} protection against {Cache-Based} side channel attacks in the cloud," in *21st USENIX Security Symposium (USENIX Security 12)*, 2012, pp. 189–204.

[118] F. Liu, Q. Ge, Y. Yarom, F. Mckeen, C. Rozas, G. Heiser, and R. B. Lee, "Catalyst: Defeating last-level cache side channel attacks in cloud computing," in *2016 IEEE international symposium on high performance computer architecture (HPCA)*. IEEE, 2016, pp. 406–418.

[119] P. Li, D. Gao, and M. K. Reiter, "Stopwatch: a cloud architecture for timing channel mitigation," *ACM Transactions on Information and System Security (TISSEC)*, vol. 17, no. 2, pp. 1–28, 2014.

[120] A. Chen, W. B. Moore, H. Xiao, A. Haeberlen, L. T. X. Phan, M. Sherr, and W. Zhou, "Detecting covert timing channels with {Time-Deterministic} replay," in *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, 2014, pp. 541–554.

[121] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 623–656, 1948.

Appendix A. Responsible Disclosure Timeline

We responsibly disclose our findings to all the affected web browsers and the detailed timeline is provided as follows.

- 2024/9/6, earlier that day: We responsibly disclosed our findings to four browsers, including Chrome, Firefox, Edge, and Opera.
- 2024/9/6, later that day: We received responses from Edge and Opera developers, who confirmed that they received our message. We did not hear from them further.
- 2024/9/6, later that day: We received responses from Chrome developers, who later decided not to patch due to the difficulty of deploying a practical defense.
- 2024/9/8, earlier that day: We responsibly disclosed our findings to Tor Browser developers.
- 2024/9/8, later that day: Tor Browser developers replied to us and said that they would follow up with Firefox developers.
- 2024/9/11: Firefox developers replied to us and confirmed our attack after a few exchanges. They worked with us for a practical defensive patch, but we did not find a mutually agreed-upon solution. For example, we find that small jitters cannot mitigate our attack. We agreed to continue to work on a solution if we find one.
- 2025/8/19: We responsibly disclosed our findings to Safari browser developers.
- 2025/8/20: We received responses from Safari developers. They reproduced our attack and are planning to address it.
- 2025/9/20: Given the complexity of our attack, Safari developers have informed us that they require sufficient time for their fix, which will not be completed until Spring 2026. Due to this ethical consideration, this paper will remain embargoed until the first day of the conference.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.

Appendix B.
Meta-Review

B.1. Summary

The paper identifies GPU command queue contention as a covert channel between browsers and browser modes. The paper develops a framework, called C3F, that facilitates communication over this covert channel. The evaluation shows that C3F over the GPU command queue enables higher bandwidth communication than prior covert channel attacks on browsers exploiting event loops and rendering contention.

B.2. Scientific Contributions

- Creates a New Tool to Enable Future Science
- Identifies an Impactful Vulnerability
- Provides a Valuable Step Forward in an Established Field

B.3. Reasons for Acceptance

1) The paper identifies a browser covert channel based on GPU command queue contention, and it demonstrates that this channel has broader scope and enables more reliable communication than those identified in prior work.
2) The C3F framework will be open-sourced and has the potential to be useful for future covert channels analysis.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:26:08 UTC from IEEE Xplore. Restrictions apply.